

APIOBike

APIOBike — это недавно запущенный сервис проката велосипедов в городе APIO. В городе установлено несколько станций, что позволяет пользователям брать или возвращать велосипеды на любой станции. Благодаря удобству и низкой стоимости, он быстро стал самым важным видом транспорта в городе APIO. Однако APIOBike столкнулся с общей проблемой всех сервисов проката велосипедов: дисбаланс между количеством взятых и возвращенных велосипедов на разных станциях. Это приводит к тому, что на некоторых станциях очень мало велосипедов, и жители рядом не могут их взять, в то время как другие станции переполнены велосипедами, которые местным жителям не нужны.

Чтобы решить эту проблему, APIOBike планирует каждую ночь отправлять грузовик для перераспределения велосипедов между станциями, чтобы на каждой станции было подходящее количество велосипедов. Всего в городе APIO N станций, пронумерованных $0, 1, \dots, N - 1$. В ходе наблюдений APIOBike выяснил, что каждый вечер количество велосипедов на станции i всегда равно фиксированному значению $A[i]$, и ночью никто не берет и не возвращает велосипеды. Компания стремится к тому, чтобы каждое утро на станции i было ровно $B[i]$ велосипедов.

Эти N станций соединены $N - 1$ выделенными дорогами для грузовика-перераспределителя. Каждая дорога соединяет две различные станции, и грузовик может перемещаться только по этим дорогам. Каждая дорога имеет длину, равную единице. Сеть выделенных дорог, то есть грузовик может добраться между любыми двумя станциями по сети выделенных дорог. Каждую ночь APIOBike должен отправить грузовик-перераспределитель, чтобы на каждой станции i было ровно $B[i]$ велосипедов. Грузовик может начать свой маршрут на любой станции и закончить его на любой станции.

В начале перераспределения грузовик пуст. Когда грузовик находится на станции, водитель может загрузить в грузовик любое количество велосипедов со станции или выгрузить любое количество велосипедов из грузовика на станцию. Грузовик и станции имеют неограниченную вместимость для велосипедов, но количество велосипедов в любом месте никогда не должно становиться отрицательным. Компания хочет определить минимально возможное расстояние перемещения для завершения перераспределения и соответствующую стратегию.

Детали реализации

Вы должны реализовать следующую функцию:

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : массив длины N , где $A[i]$ — количество велосипедов на станции i каждый вечер.
- B : массив длины N , где $B[i]$ — требуемое количество велосипедов на станции i каждое утро.
- U, V : массивы длины $N - 1$, представляющие дорожную сеть. Для каждого $0 \leq i < N - 1$, i -я дорога соединяет станцию $U[i]$ и станцию $V[i]$.
- Эта функция вызывается **не более 150 000 раз** для каждого тестового примера.

Функция должна вернуть пару массивов (X, Y) одинаковой длины $k + 1$, представляющую стратегию перераспределения:

- X : последовательность индексов посещенных станций в хронологическом порядке.
- Y : количество загруженных велосипедов на каждой посещенной станции. Для каждого $0 \leq j \leq k$:
 - Если $Y[j] \geq 0$, водитель выгружает $Y[j]$ велосипедов на станции $X[j]$ из грузовика, увеличивая количество велосипедов станции на $Y[j]$.
 - Если $Y[j] < 0$, водитель загружает $-Y[j]$ велосипедов со станции $X[j]$ в грузовик, уменьшая количество велосипедов станции на $-Y[j]$.

Допустимая стратегия перераспределения (X, Y) с расстоянием перемещения k должна удовлетворять следующим условиям:

- Для каждого $0 \leq j \leq k$, $0 \leq X[j] < N$.
- Для каждого $0 \leq j < k$, станции $X[j]$ и $X[j + 1]$ напрямую соединены дорогой.
- Для каждого $0 \leq j \leq k$, $\sum_{t=0}^j Y[t] \leq 0$.
- Для каждой станции $0 \leq i < N$ и каждого шага $0 \leq j \leq k$, пусть $S_{i,j}$ — сумма $Y[t]$ по всем шагам t таким, что $0 \leq t \leq j$ и $X[t] = i$.
 - Количество велосипедов на станции i никогда не падает ниже нуля: $A[i] + S_{i,j} \geq 0$.
 - После завершения стратегии на каждой станции i должно быть ровно $B[i]$ велосипедов: $A[i] + S_{i,k} = B[i]$.

Ограничения

- $2 \leq N \leq 300\,000$
- Сумма N по всем вызовам `find_rebalancing_strategy` не превышает 300 000 в каждом тестовом примере.
- $0 \leq U[i], V[i] < N$ и $U[i] \neq V[i]$ для каждого i такого, что $0 \leq i < N$.
- $0 \leq A[i], B[i] \leq 10^9$ для каждого i такого, что $0 \leq i < N$.
- Между любой парой станций можно перемещаться по сети дорог.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- Существует хотя бы один i такой, что $0 \leq i < N$ и $A[i] \neq B[i]$.

Подзадачи и оценка

Здесь T — количество вызовов `find_rebalancing_strategy`, а $\sum N$ — сумма всех N по всем вызовам `find_rebalancing_strategy`.

Подзадача	Оценка	Дополнительные ограничения
1	4	Станции и дороги удовлетворяют свойству Р (см. ниже). Существует ровно один i такой, что $0 \leq i < N$ и $A[i] > 0$.
2	11	$T \leq 10$. $N \leq 7$. Станции и дороги удовлетворяют свойству Р.
3	16	Станции и дороги удовлетворяют свойству Р.
4	9	Существует ровно один i такой, что $0 \leq i < N$ и $A[i] > 0$.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	Нет дополнительных ограничений.

Свойство Р: Для каждого $0 \leq i < N - 1$, $U[i] = i$ и $V[i] = i + 1$. Для каждого $0 \leq i < N$, $A[i] \neq B[i]$.

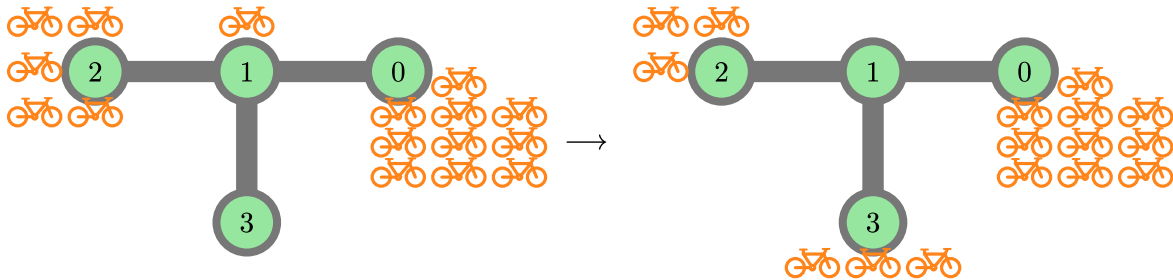
Если возвращаемые массивы X и Y имеют длину ровно $k^* + 1$, где k^* — минимально возможное расстояние перемещения, но (X, Y) не является допустимой стратегией, вы получите 50% от оценки.

Пример

Пример 1

Рассмотрим следующий вызов:

```
find_rebalancing_strategy(4,
                          [10, 1, 5, 0],
                          [10, 0, 3, 3],
                          [0, 1, 1],
                          [1, 2, 3])
```



В городе АРГО $N = 4$ станции. Изначально на станциях $A = [10, 1, 5, 0]$ велосипедов, цель — иметь на каждой станции $B = [10, 0, 3, 3]$ велосипеда.

Предположим, грузовик-перераспределитель начинает на станции 2 и выполняет следующие шаги:

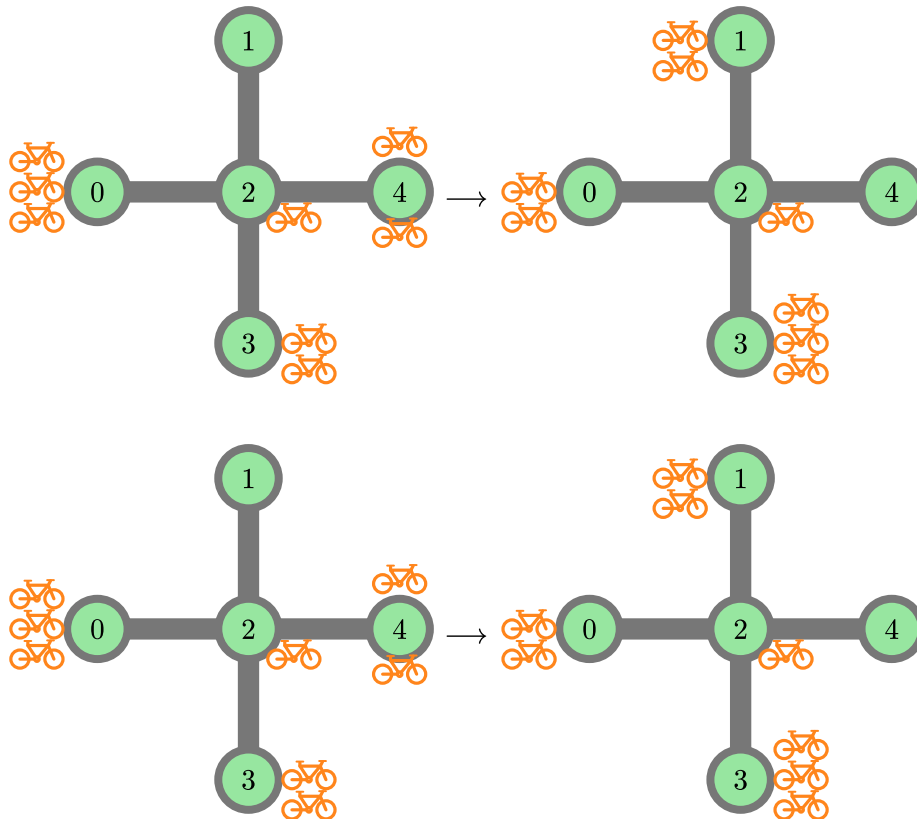
- На станции 2 загрузить 2 велосипеда в грузовик. На станции 2 становится 3 велосипеда, в грузовике 2 велосипеда.
- Переместиться на станцию 1 и загрузить 1 велосипед в грузовик. На станции 1 становится 0 велосипедов, в грузовике 3 велосипеда.
- Переместиться на станцию 3 и выгрузить 3 велосипеда. На станции 3 становится 3 велосипеда, в грузовике 0 велосипедов.

Общее расстояние перемещения равно 2. Соответствующие возвращаемые массивы для этой стратегии: $X = [2, 1, 3]$ и $Y = [-2, -1, 3]$. Можно доказать, что эта стратегия достигает минимального расстояния. Таким образом, функция должна вернуть $([2, 1, 3], [-2, -1, 3])$.

Пример 2

Рассмотрим следующий вызов:

```
find_rebalancing_strategy(5,
                          [3, 0, 1, 2, 2],
                          [2, 2, 1, 3, 0],
                          [2, 2, 2, 2],
                          [0, 4, 3, 1])
```



В городе APIO $N = 5$ станции. Изначально на станциях $A = [3, 0, 1, 2, 2]$ велосипедов, цель — иметь на каждой станции $B = [2, 2, 1, 3, 0]$ велосипеда.

Предположим, грузовик-перераспределитель начинает на станции 0 и выполняет следующие шаги:

- На станции 0 загрузить 1 велосипед.
- Переместиться на станцию 2 и загрузить 1 велосипед.
- Переместиться на станцию 1 и выгрузить 2 велосипеда.
- Переместиться на станцию 2.
- Переместиться на станцию 4 и загрузить 2 велосипеда.
- Переместиться на станцию 2 и выгрузить 1 велосипед.
- Переместиться на станцию 3 и выгрузить 1 велосипед.

Общее расстояние перемещения равно 6. Соответствующие возвращаемые массивы для этой стратегии: $X = [0, 2, 1, 2, 4, 2, 3]$ и $Y = [-1, -1, 2, 0, -2, 1, 1]$. Можно доказать, что эта

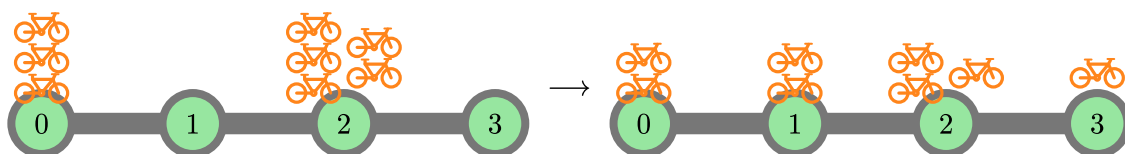
стратегия достигает минимального расстояния. Таким образом, функция должна вернуть $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$.

Другие допустимые стратегии с расстоянием 6, такие как $X = [0, 2, 3, 2, 4, 2, 1]$ и $Y = [-1, 0, 1, 0, -2, 0, 2]$, также считаются правильными. Возврат массивов X, Y длины $7 = 6 + 1$, но не являющихся допустимой стратегией, например $X = Y = [0, 0, 0, 0, 0, 0, 0]$, принесет 50% от оценки.

Пример 3

Рассмотрим следующий вызов:

```
find_rebalancing_strategy(4,  
                           [3, 0, 5, 0],  
                           [2, 2, 3, 1],  
                           [0, 1, 2],  
                           [1, 2, 3])
```



Оптимальное решение: $X = [2, 1, 0, 1, 2, 3]$ и $Y = [-1, 1, -1, 1, -1, 1]$. Функция должна вернуть $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$. Другие допустимые стратегии, такие как $X = [2, 1, 0, 1, 2, 3]$ и $Y = [-2, 2, -1, 0, 0, 1]$, также считаются правильными.

Этот пример удовлетворяет ограничениям подзадач 2 и 3.

Грейдер для участников

Первая строка ввода должна содержать одно целое число T — количество сценариев. Далее следует описание T сценариев, каждый в формате, указанном ниже.

Формат ввода:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

Формат вывода:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```